SMART LINE MES · ④ 설비보전 AI · 김영창

설비보전 AI 학습 보고서

고장 보고 문장 → 담당 분야 · 조치 시점 자동 분류 · 최종 모델 v5 (TF-IDF+LR · BiLSTM · KLUE-RoBERTa 앙상블) · 작성 2026.10.08

89.1%	87.0%	4건	1,200
담당 분야 정확도	조치 시점 정확도	오경보 (즉시 아닌데 즉시)	학습 문장
최종 시험 92문장 (v1 80.4%)	최종 시험 92문장 (v1 73.9%)	v1 10건	상황 300 × 말투 4

요약

- **문제** — 고장 보고가 말 · 메모로 자유롭게 전달돼 "누가, 언제 고칠지" 정하는 데 시간이 걸린다. 문장 한 줄로 **담당 분야 4종**과 **조치 시점 3단계**를 자동으로 정한다.
- **v1의 실패** — 모델을 바꿔도(KLUE · 임베딩 · 기준값 조정) 사람 문장의 긴급도 정확도가 오르지 않았다. 원인은 데이터: 라벨을 먼저 정하고 문장을 생성해 **말투가 긴급도를 대신**했다 (과장하면 즉시).
- **v5의 해법** — 기준표 v2 → 상황 300개 설계 → 규칙으로 라벨 계산 → **정답을 숨긴 채** 4가지 말투로 문장화. 학습(Claude)과 평가(GPT · Gemini · 사람) 생성기를 분리하고, 최종 시험은 봉인 후 1회만 평가.
- **결과** — 담당 80.4 → 89.1%, 조치 시점 73.9 → 87.0%, 오경보 10 → 4건. 다만 사람이 쓴 문장에서는 담당 70% · 조치 65%로 여전히 낮아, 현장 피드백 수집 → 재학습이 다음 과제.

1. 문제 정의

구분	내용
입력	작업자가 쓰거나 말로 남긴 고장 보고 문장 1줄 (예: "프레스 내려올 때 끼익 소리 계속 남")
출력 ① 담당 분야	누가 고칠지 — **기계 정비 · 전기·제어 · 유공압 · 센서·계측** (코드값 기계 · 전기 · 유압공압 · 센서)
출력 ② 조치 시점	언제 고칠지 — **즉시 조치 · 당일 조치 · 정기 점검** (코드값 상 · 중 · 하)
부가 출력	위험 단어 규칙 적용 여부 · 관리자 확인 필요 여부와 이유 · 협업 필요 담당
대상 설비	프레스 #1 · 컨베이어 · 로봇 적재기 · 비전 검사기 · 자재 투입기 (5종)
목표 (기획서)	고장 보고 유형 분류 정확도 80% 이상 · 긴급 고장 시 설비 자동 정지 및 정비 이력 체계화
일정	9/29 ~ 10/2 모델 제작 (기준표 v1 9/30 → v2 10/1 → 최종 시험 10/2), 10/5 ~ 10/7 웹 통합

2. 라벨 기준 — 기준표

2-1. 변경 이력

버전	날짜	내용
v1	2026-09-30	유형 4종(기계 · 전기 · 유압공압 · 센서), 긴급도 상 · 중 · 하
v1.1	2026-10-01	'중'에서 "악화 가능성" 삭제 — 가능성만으로는 '하'

v2	2026-10-01	긴급도 → **조치 시점**, 유형 → **담당 분야**(고칠 1차 담당자). 상황 속성 5종과 자동 판정 규칙, 경계 부품 판정표, 애매 · 범위 밖 처리 추가. **확정 후 변경 금지** (바꾸려면 v3로 새로 만들고 모든 데이터 재판정)

2-2. 상황 속성 5종 → 조치 시점 자동 판정

속성	값	정의 요지
안전 위험	있음 / 없음	**지금** 사람이 다칠 수 있는 요인 노출 (화재 · 연기, 노출 도체, 바닥 기름, 부품 비산, 안전장치 위험 방향 이상)
가동 상태	정상 / 저하 / 정지	정지 = 재가동 불가 · 핵심 기능 불가 / 저하 = 속도 저하 · 재조작 · 간헐 정지
품질 영향	있음 / 없음	불량 발생 · 유출 위험 · 검사 신뢰도 저하
빈도	1회성 / 간헐 / 지속	1회성 = 이미 조치되어 지금 정상
추세	변화없음 / 악화	문장에 악화가 **드러날 때만** (점점, 늘었음, 심해짐)

판정 규칙 (위에서부터 먼저 해당하는 것)

1. **즉시 조치(상)** — 안전 위험 있음 또는 가동 정지
2. **당일 조치(중)** — 품질 영향 있음 · 가동 저하 · 추세 악화 (단, 1회성이면 제외)
3. **정기 점검(하)** — 그 외

보조 원칙

- 가능성만으로는 앞당기지 않음 (안전은 현재 노출 여부)
- 복수 고장은 가장 급한 것 기준
- 고위험 설비(프레스 · 로봇)는 애매하면 한 단계 앞당김 — 평가 데이터 라벨링에만

사례 — 누유: 즉시 바닥 고임 · 당일 계속 떨어짐 · 정기 맺힘 / 비전: 즉시 검사 불가 · 당일 오판정 반복 · 정기 화면 품질만 저하

2-3. 담당 분야 판정

- 원칙: 겉 증상이 아니라 **고장을 고칠 1차 담당자**로 판단. 원인 불명이면 먼저 진단할 담당 (알람 · 전원 → 전기, 감지 · 인식 → 센서, 소리 · 진동 · 움직임 → 기계).
- 경계 부품 판정표: 모터(소음 · 진동 · 베어링 = 기계 / 안 켜짐 · 과열 트립 · 타는 냄새 = 전기), 그리퍼(기구 마모 = 기계 / 공압 누설 · 흡착력 = 유공압 / 파지 센서 = 센서), 안전장치(스위치류 = 전기 / 감지기류 = 센서), 신호 이상(배선 · PLC · 통신 = 전기 / 센서 본체 · 감도 · 오염 = 센서).
- 라벨링 대상 아님: 정보 부족(애매), 범위 밖(자재 부족 · 청소 요청 · 질문) → 학습 · 채점 제외, 시스템에서는 관리자 확인.
- 현장 용어 목록: 설비별 부품 · 대표 증상, 현장 표현(끼익 · 덜컹덜컹 · 쉭쉭, 컨베어 · 모타 · 실린다 · 센사). 규칙 — **표현의 강도와 조치 시점은 서로 무관하게** (정기 점검에도 "엄청 시끄러움", 즉시에도 담담한 말투).

3. 데이터셋

3-1. 데이터를 다시 만든 이유

> "v4까지의 실험에서 확인한 데이터 문제 — **라벨 먼저 생성 → 말투가 긴급도를 대신함, 단일 생성기, 기준 변경, 현장 어휘 부족** — 를 해결하기 위해 기준표 v2로 데이터를 처음부터 다시 만든다." (v5 README)

3-2. 구성 — 학습과 평가의 생성기를 분리

데이터셋	만든 방법	분량	용도
학습용 상황	Claude가 상황 속성 설계	300개	담당 4 × 조치 3 × 25개. 규칙으로 라벨 자동 계산
학습 · 검증 문장	Claude가 상황 × 말투 4종으로 문장화	1,200문장	train 1,020 / val 180 (상황 단위 15% 분리)
선택용 dev	GPT 36 + Gemini 36 (새 대화창)	72문장	앙상블 비중 · 설정 선택 (즉시 · 당일 · 정기 각 24)
최종 시험	GPT 36 + Gemini 36 + 직접 작성 20	92문장	**봉인** — 마지막에 한 번만 평가

원칙: 학습 생성기(Claude)는 평가 데이터에 쓰지 않는다. 직접 작성 20문장은 학습 데이터와 AI 문장을 보기 전에, AI 도움 없이 작성자가 담당 · 조치 시점 항목에 맞게 상황을 직접 정해 썼다. 생성 프롬프트 원문 요약은 부록 A.

3-3. 만드는 과정 (8단계)



핵심: 평가 데이터(②③)를 학습 데이터(⑤⑥)보다 **먼저** 만들고 봉인 → 학습 데이터를 만들면서 시험 문제에 맞춰 버리는 일을 막음.

생성기의 정답(target)은 쓰지 않고, 평가 데이터도 **기준표 v2 규칙으로 라벨을 다시 계산** (다르면 규칙 값 사용).

학습 문장을 쓰는 LLM에게는 상황 설명만 주고 조치 시점 · 속성은 보여 주지 않음 → 말투가 정답을 따라가지 않게.

3-4. 단계별 처리와 품질 관리

단계 (스크립트)	처리 내용
상황 설계 `build_scenarios.py`	열 = 설비 · 부품 · 담당 · 속성 5종 · 상황 설명. 값 검사 → 같은 설명 중복 제거 → 규칙으로 조치 시점과 근거 계산 → 분포 점검(담당 × 조치 칸마다 25개 목표) → 한 담당 안에서 12% 넘는 부품 경고 → ID 부여 (S0001~)
상황 검수 `check_scenarios.py`	설명과 속성이 어긋나 보이는 행을 9가지 규칙으로 표시 (타는 냄새인데 안전 없음, 바닥 기름인데 안전 없음, 점점 늘어남인데 악화 아님, 모터 소음인데 전기 등) → 300개 중 19개 의심 → 사람이 기준표로 판단해 고칠 것만 수정 (재실행 시 16개 남음)
문장화 묶음 `make_batches.py`	300개를 25개씩 12묶음, 묶음마다 담당 · 조치 시점을 고르게 섞음 (한 묶음이 즉시만 있으면 말투가 심각도에 끌려감)
정제 · 분할 `build_train.py`	말투 4종(담담 · 과장 · 보고 · 구어) × 300 = 1,200. 잘못된 ID · 말투 · 빈 문장 · 중복 제거. **정답 노출 단어 제거**(긴급, 즉시, 당일, 정기 점검, 담당, 안전 위험, 품질 영향). **지름길 점검**: 조치 시점별 평균 길이와 감정 표현(! ㅠ 엄청 완전 대박 큰일) 비율이 비슷한지. 상황 단위 층화 분할(val 15%, SEED 42) — 같은 상황 문장은 한 쪽에만
평가 데이터 `build_test_final.py`	GPT · Gemini 원본 + 수정본 + 사람 작성 합치기 → 정답 노출 단어 · ID 중복 검사 → 규칙으로 라벨 재계산 → 생성기별 담당 × 조치 조합당 3개로 균형(사람 20문장은 전부 사용). dev에서 최종 시험과 같은 문장 제거

3-5. 학습 데이터 정제 결과 (실제 파일로 확인)

단계	결과
문장화 원본	train_01~12 · 1,200행 (묶음당 96~104)
지름길 수정	**26문장 다시 씀** (train_fix.csv) — 전부 '정기 · 과장' 문장
중복 제거	같은 상황 · 말투 26행 → 고친 문장(train_fix)을 남김
정답 노출 단어	0행
말투 완비	상황 300개 모두 4가지 말투
최종	**1,200문장** → train 1,020 (255상황) / val 180 (45상황)
문장 길이	평균 50.2자 (16 ~ 124) · 담담 26.6 · 구어 44.1 · 과장 55.6 · 보고 74.3

지름길 점검	즉시	당일	정기
감정 표현 비율 (수정 전)	25.8%	25.2%	**18.4%**
감정 표현 비율 (수정 후)	25.8%	25.2%	**25.0%**
평균 길이 (수정 전 → 후)	52.1	51.5	45.2 → 46.8

정기 점검 문장에만 감정 표현(! · ㅠ · 엄청 · 진짜 등)이 적어 "담담하면 정기"라는 지름길이 생길 수 있었다. 정기 · 과장 26문장을 더 요란하게 다시 써서 세 조치 시점의 비율을 맞췄다. 말투 × 조치 시점은 말투마다 즉시 100 · 당일 102 · 정기 98로 같다.

담당 × 조치 시점	상황 300			train 1,020			val 180		
	즉시	당일	정기	즉시	당일	정기	즉시	당일	정기
기계	25	25	26	84	88	88	16	12	16
전기	25	27	22	84	92	76	16	16	12
유압공압	25	25	25	84	84	88	16	16	12
센서	25	25	25	84	84	84	16	16	16

칸마다 25개를 목표로 설계했지만 조치 시점을 LLM이 아니라 규칙으로 다시 계산해 몇 칸이 22~27개로 달라졌다. 설비는 5종 각 60개로 균등. 상황 검수는 처음 19개 의심 행 → 수정 후 다시 실행 시 16개 — 남은 행은 기준표로 판단해 그대로 둔 것(규칙에 걸려도 모두 오류는 아님).

3-6. 최종 시험 92문장 분포

구분	즉시(상)	당일(중)	정기(하)	기계	전기	유압공압	센서
GPT 36	12	12	12	9	9	9	9
Gemini 36	12	12	12	9	9	9	9
사람 20	9	8	3	7	5	4	4
합계 92	**33**	**32**	**27**	**25**	**23**	**22**	**22**

말투: 과장 33 · 담담 31 · 보고 28 / 설비: 컨베이어 22 · 프레스 21 · 자재 투입기 17 · 비전 검사기 16 · 로봇 적재기 16.
봉인 규칙: 생성 후 형식만 확인하고 문장은 읽지 않음 → 모델 선택에 쓰지 않음 → 마지막에 1회 평가. `final_test.py`는 실행 시 잠금 파일(v5_final_test.done)을 만들고, 다시 돌리려면 `--force`가 필요하며 그 결과는 최종 시험으로 보고하지 않음.

4. 전처리

대상	처리
공통 토큰화	**kiwipiepy(Kiwi)** 형태소 분석. 문장부호(S*)만 제거, 영어 · 숫자 · 한자와 조사 · 어미는 유지. 정해진 라벨 값인 행만 사용
TF-IDF	형태소 1~2gram + 글자(char_wb) 2~4gram 두 특징을 합침 — 글자 단위는 오타 · 줄임말(컨베어, 모타)에 강함
BiLSTM	학습 문장 형태소로 단어장(빈도순, <PAD>=0, <OOV>=1). 최대 길이 = 토큰 수 95백분위(10~60 사이). **v5: 양방향 앞 패딩** — 정방향 [빈칸…, w1…wn], 역방향 [빈칸…, wn…w1]. 두 LSTM 모두 실제 단어를 마지막에 읽어 마지막 상태가 실제 단어로 결정됨 (v1은 뒤 패딩 + mask_zero였는데, Keras 버전에 따라 마스크가 꺼져 불리했음)
KLUE-RoBERTa	klue/roberta-base 토크나이저, max_length 64, truncation, 배치 단위 padding
서비스 입력	앞뒤 공백 제거만. 맞춤법 · 줄임말 변환은 하지 않음 (학습 데이터에 현장 표현을 넣어 대응)

5. 모델 3종

세 모델 모두 같은 문장에서 **담당(4)**과 **조치 시점(3)** 두 가지를 동시에 냅니다. 학습 환경: Windows · uv 가상환경 · **CPU** (KLUE 약 10분).

항목	TF-IDF + 로지스틱 회귀	BiLSTM	KLUE-RoBERTa
구조	형태소 · 글자 TF-IDF 결합 → LogisticRegression (담당 · 시점 각각 1개)	Embedding 128 → 정방향 · 역방향 LSTM 64 → 결합 → Dropout 0.3 → Dense 64 → Dropout 0.3 → 출력 2개(softmax 4, 3)	사전학습 인코더 → [CLS] → Dropout 0.1 → 출력 헤드 2개(4, 3)
설정	C=5.0 · class_weight=balanced · max_iter 3000	Adam 1e-3 · 최대 30 epoch · batch 32 · EarlyStopping(val_loss, patience 3) · SEED 42	AdamW 2e-5 · weight decay 0.01 · 6 epoch · batch 16 · warmup 10% · grad clip 1.0 · 손실 = CE(담당) + CE(시점)
epoch 선택	—	val_loss 조기 종료	dev에서 선택 규칙으로 가장 좋은 epoch → **4 epoch**
저장된 설정 (meta.json)	baseline.joblib	단어장 **1,443개**(<PAD> · <OOV> 포함) · 최대 길이 **41** 토큰 · 입력 two_way_prepad · seed 42	klue/roberta-base · max_len 64 · best_epoch 4 · seed 42
장점	빠르고 단어 근거가 분명함	어순 · 문맥 반영	한국어 사전학습 — 표현이 달라도 의미 파악
dev 단독 점수 (담당 / 조치 / 오경보)	0.833 / 0.806 / 9	0.847 / 0.764 / — (즉시 탐지 0.833으로 탈락)	0.889 / 0.903 / 3

6. 실험 과정 — v1에서 v5까지

v1 ~ v4 — 같은 초기 데이터로 고도화를 시도한 단계
초기 데이터셋(AI에게 라벨을 정해 주고 프롬프트로 문장을 생성)을 바탕으로, **데이터를 보강하거나 모델을 바꿔 보는 방식**으로 성능을 끌어올리려 했다. 그러나 사람 문장의 긴급도 정확도가 근본적으로 오르지 않아, v5에서는 **데이터를 처음부터 다시 설계 · 정제 · 전처리한 뒤** 학습했다. v2 ~ v4 각 회차의 변경 내용과 점수는 기록이 남아 있지 않아, 아래는 v1의 최종 평가와 그 이후 모델 교체 실험 결과로 정리한다.

6-1. v1 데이터와 학습 (9/29 ~ 10/1)

항목	내용 (legacy_v1 폴더에서 확인)
만든 방법	Claude에게 **담당 × 긴급도 조합을 먼저 정해 주고** 문장을 생성 — 조합당 2파일 × 약 50문장 = 24파일 (예: gen_claude_기계_상_1.csv). 같은 부품을 세 가지 심각도로 쓴 **대조 문장** 150개 추가 (예: 오일 얼룩 조금 → 하 / 계속 떨어짐 → 중 / 확 새서 바닥 미끄러움 → 상)
클래스	담당 4종(기계 · 전기 · 유압공압 · 센서) × 긴급도 3단계(상 · 중 · 하) — v5와 같은 구성
정제 · 분할	merge.py: 라벨 값 검사 · 2글자 미만 · 중복 제거 → **1,307문장** → 담당 × 긴급도 층화 분할 → train **1,110** / val **197** (문장 단위 분할)
평가 데이터	test_llm 59문장(GPT, 설정 선택용) · test_human 54문장 (Gemini 43 + Claude 11 — 파일에는 source=user로 표기됐지만 실제로는 Gemini가 작성)

BiLSTM 학습	단어장 1,735 · 최대 길이 27 · 7 epoch (조기 종료). 마지막 epoch 학습 정확도 담당 0.986 · 긴급도 0.973 vs 검증 최고 담당 0.959 · **긴급도 0.817** (4 epoch) — 학습 문장은 거의 다 맞히지만 긴급도는 검증에서 크게 떨어짐
최종 모델	TF-IDF+LR 0.75 + BiLSTM 0.25 (models_v4 = 이전 작업 em_model_v1 안에서 v1~v4 실험을 반복한 마지막 모델)

6-2. v1 평가 결과

모델 (test_human 54문장, 위험 단어 규칙 포함)	범위	유형	긴급도	상 재현	오경보
최종 앙상블	전체 54	0.926	0.796	0.875	7
최종 앙상블	**Gemini 작성 43**	0.907	**0.744**	0.870	7
기준 모델 단독	Gemini 작성 43	0.860	0.698	0.913	10
BiLSTM 단독	Gemini 작성 43	0.814	0.698	0.826	8

Gemini 작성 43문장 긴급도 macro F1 0.64 — '중'을 '상'으로 올리는 오류가 많았고('하' 3문장 중 1개만 맞음), 과장된 표현에 끌려가는 경향이 보였습니다.

6-3. 모델을 바꿔 본 실험 — 효과 없음

실험	결과 (Gemini 작성 43문장 긴급도)
KLUE-RoBERTa 미세조정 + 앙상블	0.698 (기존 0.744보다 낮음), 오경보 11
문장 임베딩(ko-sroberta) + LR	선택 데이터 기준 비중 0.0 → 추가 안 함
'상' 판정 기준값 조정	기준값 조정 없음이 선택, 0.698 · 오경보 11

결론 — 모델을 바꾸거나 키워도 사람 문장의 긴급도가 오르지 않았다 → **원인은 모델이 아니라 데이터**. 라벨을 먼저 정하고 문장을 생성해 "과장 = 즉시, 담담 = 정기"라는 지름길을 학습했다. → v5에서 데이터를 처음부터 다시 설계.

6-4. v5에서 바꾼 것

문제 (v1)	해결 (v5)
라벨 먼저 → 말투가 정답	상황 속성 먼저 설계 → 규칙으로 라벨 → 정답을 숨기고 4가지 말투로 문장화 → 말투와 조치 시점이 무관해짐
단일 생성기	학습 Claude / 평가 GPT · Gemini · 사람으로 분리
기준이 바뀜	기준표 v2 확정 후 변경 금지, 모든 데이터를 같은 규칙으로 라벨
현장 어휘 부족	현장 용어 목록(부품 · 증상 · 줄임말)을 상황 설계와 문장화에 사용
평가 데이터를 여러 번 봄	dev로만 선택, 최종 시험은 봉인 후 1회
문장 단위 분할 (같은 상황의 다른 표현이 학습 · 검증에 섞임)	상황 단위 분할 — 같은 상황의 4문장은 한쪽에만
BiLSTM 마스크 문제	양방향 앞 패딩으로 바꿔 Keras 버전과 무관하게
KLUE 효과 없음	데이터를 바꾼 뒤 KLUE를 앙상블에 정식 포함 (dev 단독 최고)

성능 비교는 v1 최종 모델(TF-IDF 0.75 + BiLSTM 0.25)과 v5를 같은 dev · 최종 시험에서 나란히 평가한 값입니다.

7. 앙상블 선택과 최종 결과

7-1. 선택 규칙 — 결과를 보기 전에 확정 (10/1)

1. **즉시 조치 탐지율 ≥ 0.9**인 조합만 후보 (급한 고장을 놓치면 안 됨)
2. 그중 조치 시점 정확도 최고 → 3. 오경보 최소 → 4. 담당 정확도 최고 → 5. 쓰는 모델 수가 적은 조합 → 6. 기준 모델 비중이 큰 조합

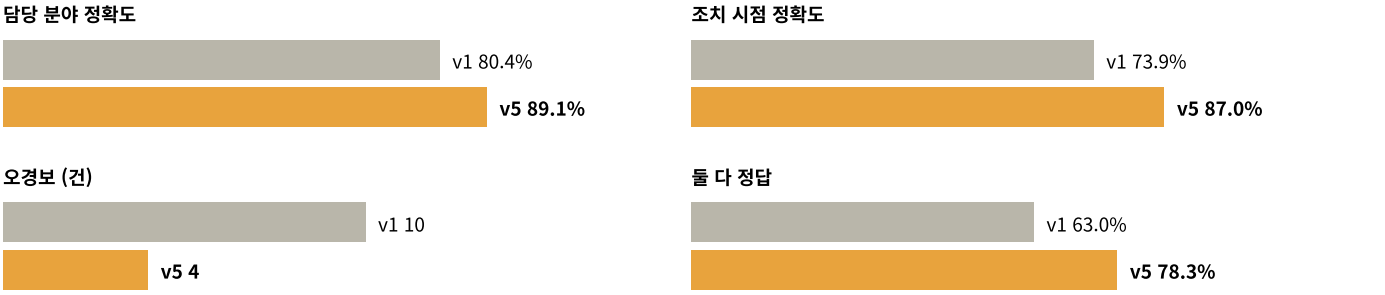
세 모델 확률을 0 · 0.25 · 0.5 · 0.75 · 1.0 비중으로 섞되 합이 1인 **15개 조합**을 dev 72문장에서 비교 → **TF-IDF 0.25 + BiLSTM 0.25 + KLUE 0.5** 선택.

오경보 = 정답이 즉시가 아닌데 즉시로 판단한 수 (불필요한 설비 정지로 이어짐)

dev 72문장	담당	조치	둘 다	즉시 탐지	오경보	과장된 정기 점검	담담한 즉시 조치
v5 앙상블	0.917	0.917	0.833	0.917	3	1.000	1.000
v1 최종	0.833	0.806	0.681	0.917	5	**0.625**	1.000

'과장된 정기 점검' · '담담한 즉시 조치'는 말투와 정답이 엇갈리는 문장만 따로 본 정확도 — 말투 지름길을 썼는지 확인하는 지표. dev 선택은 두 번 실행했고 두 번 모두 같은 조합(0.25/0.25/0.5)이 선택됨 (1차 0.903/0.903, 2차 0.917/0.917 — 사이에 BiLSTM 재학습). 최종 모델 파일 `v5_final.json`에 저장된 dev 값은 2차 결과(담당 · 조치 0.917, 둘 다 0.833, 즉시 탐지 0.917, 오경보 3)와 같다.

7-2. 최종 시험 92문장 (2026-10-02 10:08, 1회)



데이터	모델	담당	조치	즉시 탐지	오경보	문장
전체	**v5**	**0.891**	**0.870**	0.848	**4**	92
	v1	0.804	0.739	0.879	10	
	v1 기준 모델 단독	0.826	0.772	0.848	8	
GPT	v5 / v1	0.917 / 0.806	0.889 / 0.778	0.750 / 0.833	0 / 3	36
Gemini	v5 / v1	0.972 / 0.917	0.972 / 0.806	1.000 / 1.000	1 / 5	36
사람	v5 / v1	0.700 / 0.600	0.650 / 0.550	0.778 / 0.778	3 / 2	20
과장된 정기 점검	v5 / v1	—	0.667 / 0.444	—	—	9
담담한 즉시 조치	v5 / v1	—	0.917 / 1.000	—	—	12

7-3. 클래스별 결과와 혼동행렬 (v5, 92문장)

정밀도 · 재현율 · F1은 최종 시험 예측 파일(v5_final_test_pred.csv)에서 다시 계산한 값 (정확도는 시험 로그와 일치).

담당	정밀도	재현율	F1	n
기계	1.000	0.840	0.913	25
전기	0.833	0.870	0.851	23
유압공압	1.000	0.909	0.952	22
센서	0.778	0.955	0.857	22
macro			**0.893**	92

조치 시점	정밀도	재현율	F1	n
즉시(상)	0.875	0.848	0.862	33
당일(중)	0.848	0.875	0.862	32
정기(하)	0.889	0.889	0.889	27
macro			**0.871**	92

정답 \ 예측	기계	전기	유공압	센서
기계	**21**	2	0	2
전기	0	**20**	0	3
유압공압	0	1	**20**	1
센서	0	1	0	**21**

정답 \ 예측	즉시	당일	정기
즉시	**28**	3	2
당일	3	**28**	1
정기	1	2	**24**

v1 조치 혼동행렬: [[29,3,1],[6,21,5],[4,5,18]] — 당일 · 정기를 즉시로 올리는 오류(오경보 10)가 v5에서 4로 줄었다.

7-4. 해석

- **말투 지름길이 줄었다** — 과장된 정기 점검 정확도 dev 0.625 → 1.000, 시험 0.444 → 0.667. 오경보 10 → 4.
- **담당은 '센서'로 몰리는 경향** — 센서 재현율 0.955지만 정밀도 0.778 (전기 → 센서 3건, 기계 → 센서 2건).
- **즉시 탐지율은 0.848로 선택 기준(0.9)보다 낮다** — dev에서는 0.917이었음. GPT 문장에서 0.750. v1(0.879)보다도 약간 낮아, 위험 단어 규칙 · 확인 필요 장치로 보완.
- **사람 문장이 가장 어렵다** — 담당 0.700 · 조치 0.650. 짧고 정보가 빠진 메모 형태라 AI가 만든 문장과 분포가 다름 → 현장 데이터 수집이 다음 과제.

대표 오답 (최종 시험)	정답 → 예측
프레스 #1이 정지버튼을 눌러도 멈추지 않음 (사람)	기계 · 즉시 → 전기 · 즉시
비전 카메라의 판정 오류가 반복되고 … 불량품을 놓칠 가능성 (GPT)	즉시 → 당일
금형 교체 후 체결 볼트가 풀려 … 재체결하여 현재는 정상 (GPT)	즉시 → 정기
비전 관련 PC 안켜지는데 뭐가 문제인지 (사람)	당일 → 즉시

8. 서비스 후처리 규칙

모델 확률만으로 결정하지 않고, 안전 쪽으로 보완하는 규칙을 붙였습니다.

규칙	값	동작
위험 단어	19개	연기 · 불꽃 · 불똥 · 타는 냄새 · 탄내 · 감전 · 스파크 · 누전 · 화재 · 폭발 · 침수 · 젖었 · 물이 떨어 · 물 떨어 · 터졌 · 터진 · 협착 · 끼임 · 끼였 → 모델이 즉시가 아니라고 해도 **즉시로 올림**. v1의 "불이"는 "센서 불이 깜빡"처럼 표시등을 뜻해 오경보를 만들어 제외
확신도 낮음	LOW_CONF 0.5	담당 · 조치의 최고 확률 중 작은 값이 0.5 미만 → 관리자 확인 필요
문장이 짧음	MIN_LEN 5	5글자 미만 → 확인 필요
추측 · 질문 · 요청	정규식	"?", "것 같", "모르겠", "봐 주세요", "해야 하나", "어떡" 등 → 확인 필요
협업 분야	COLLAB_GAP 0.15	1 · 2위 담당 확률 차이 < 0.15 → 2위 담당을 '협업'으로 함께 표시

확인 필요 장치 평가	dev 72 (공정)	최종 시험 92 (참고)
확인 필요로 넘긴 비율	6.9% (5문장)	13.0% (12문장)
자동 처리한 문장의 정확도 (둘 다 정답)	0.851	0.863
오답 중 확인 필요로 잡아낸 비율	23% (13개 중 3)	45% (20개 중 9)
놓친 즉시 조치 중 확인 필요로 넘어간 것	2건 중 2건	5건 중 3건
사람 문장의 확인 필요 비율	—	50%

LOW_CONF 기준값 선택 — dev 72문장 스윕

선택 규칙(실행 전에 정함): 확인 필요 비율 20% 이하인 값 중 → 자동 처리 정확도 최고 → 같으면 확인 필요 비율이 낮은 값. 최종 시험은 쓰지 않음.

LOW_CONF	확인 필요 비율	자동 처리 정확도	오답 포착률	놓친 즉시 조치 포착
0.40	0.0%	0.833	0.000	0 / 2
0.45	2.8%	0.843	0.083	0 / 2
0.50 (선택)	**6.9%**	**0.866**	**0.250**	**2 / 2**
0.55	13.9%	0.855	0.250	2 / 2
0.60	**25.0%**	0.889	0.500	2 / 2
0.65	**30.6%**	0.920	0.667	2 / 2
0.70	**37.5%**	0.933	0.750	2 / 2

0.60 이상은 정확도가 더 높지만 확인 필요가 20%를 넘어 관리자 부담이 커져 제외. 20% 이하에서 0.50이 자동 처리 정확도 최고이고, 놓친 즉시 조치 2건을 모두 확인 필요로 넘김 → **LOW_CONF = 0.5**.

'추측 · 질문 · 요청' 규칙은 최종 시험 오답을 본 뒤 만든 것이라 시험 쪽 수치는 낙관적인 참고값이고, 공정한 평가는 dev 값으로 본다.

9. 웹 연동과 재학습 순환

- **판정 코드** — 학습 폴더(EM_model)의 판정 부분만 `web/maint_model.py`로 옮김. 모델은 `tools/export_maint_model.py`로 복사하며, 학습 폴더와 같은 문장 4개 결과가 글자까지 같은지 확인. 실패하면 학습 폴더 → 더미 분류기 순으로 대체. 서버 시

작 때 미리 불러와 첫 판정 대기 제거.

- **입력** — 직접 입력 또는 "말로 보고하기"(브라우저 음성 인식). 문장에서 설비 이름 자동 감지(컨베어 · 콘베어 · 벨트 등 별칭 포함). 설비가 애매하면 티켓 만들기 전에 확인.
- **연동** — 즉시 조치 판정 → 해당 설비 '고장 정지' → 컨베이어 연동 정지. 정비 완료 후 남은 즉시 티켓이 없으면 재가동. 결과 화면에 비슷한 지난 정비 · 추천 점검 순서.
- **재학습 순환** — 정비 완료 때 정비원이 실제 담당 · 조치 시점을 입력 → feedback 테이블 → 현장 정확도 집계 → **50건 쌓이면 재학습 권장** → 학습 데이터와 같은 형식의 CSV로 내보내기. (발표 시점 수집 0건)

10. 한계와 다음 단계

한계	다음 단계
사람이 쓴 문장 조치 시점 65% · 담당 70% (20문장)	현장 고장 보고 수집 → 피드백으로 정답 확보 → 재학습
즉시 탐지율 0.848 (기준 0.9 미달)	위험 단어 · 확인 필요 장치로 보완, 현장 데이터로 즉시 사례 보강
최종 시험이 92문장으로 작음 (사람 '정기' 3문장)	독립 시험 세트 · 현장 검증 확대
담당 '센서' 쏠림	경계 부품(전기 ↔ 센서) 사례 보강
문장만 보고 판단	PLC 정지 신호 · 실가동 데이터와 결합, 진동 · 온도 기반 예지보전

배운 것 — 첫 버전은 말투에 속았다. 과장해서 쓰면 긴급하다고 판단했다. 모델을 키워도 해결되지 않았고, 원인은 데이터였다. 상황을 먼저 설계하고 정답을 숨긴 채 문장을 만들고, 평가 데이터를 먼저 봉인하자 같은 모델 구조에서도 성능이 올랐다. **AI의 성능은 모델만이 아니라 데이터와 평가 설계가 함께 결정한다.**

부록 A. 데이터 생성 프롬프트 요약

원문은 EM_model/prompts/ 폴더. 세 프롬프트 모두 기준표 v2의 속성 정의와 판정 규칙을 그대로 담고, 출력은 설명 없이 CSV만 받는다.

프롬프트	생성기 · 대화창	핵심 지시
01 최종 시험 (dev도 같은 방식)	GPT · Gemini 각각 새 대화창 (Claude 사용 금지)	역할: 오래 일한 현장 작업자. 메모마다 ① 상황 → ② 속성 5종 → ③ 메모 순서로 작성. 담당 4 × 조치 3 = 12조합 × 3개 = 36개, 설비 5종 고르게, 12개 이상에 현장 용어(풀코드 · 양수 버튼 · 인터록 · 인코더 · 레귤레이터 등). 조합마다 3개는 담담 · 과장 · 보고로 다르게 쓰고 **말투와 심각도를 일부러 엇갈리게**(정기에도 과장, 즉시에도 담담). 정답 단어 금지. 저장 후 **형식만 확인하고 내용은 읽지 않음**(봉인).
02 학습용 상황 설계	Claude · 담당 분야마다 새 대화창 (4번)	역할: 설비 보전 전문가. 작업자 문장은 쓰지 않고 **상황의 사실만** 정비 기록처럼 1~2문장. 한 번에 75개(즉시 · 당일 · 정기 각 25, 설비별 5개씩), 한 부품 8개 이하. 같은 조치 시점 안에서도 원인 조합을 섞음(즉시 = 안전 / 정지, 당일 = 품질 / 저하 / 악화, 정기 = 해결된 1회성 / 영향 없는 지속 / 영향 없는 간헐). 경계 사례 필수: 누유 정도, 안전장치 방향, 비전 오판정과 검사 불가, 이미 해결된 고장, 소리는 크지만 영향 없는 고장, 조용하지만 기능을 잃은 고장. **조치 시점은 LLM이 아니라 코드가 규칙으로 계산**.
03 학습용 문장화	Claude · 대화창당 최대 3묶음	역할: 현장 메모를 재현하는 작가. 상황 하나를 담담 · 과장 · 보고 · 구어 4개 메모로. **정답(조치 시점 · 담당)은 보여 주지 않음** — 알면 정답에 맞춰 말투를 바꾸기 때문. 사실(위험 · 가동 · 품질 · 빈도 · 추세)은 바꾸지 않고 작업자 말로 바꿔 씀. 말투는 심각도와 무관(사소해도 과장은 요란하게, 심각해도 담담은 건조하게). 정답 단어 금지, 설비명은 일부 메모에서만, 현장 표현 목록(끼익 · 덜컹덜컹 · 먹통 · 탄내 · 줄줄 · 컨베어 · 모타 등)을 반복 없이. 길이: 담담 5~25자, 과장 · 구어 15~50자, 보고 30~80자.

근거 자료 — 기준표 v2 · 현장 용어 · 프롬프트 3종 · 학습/평가 데이터 원본과 정제본 · 학습 스크립트 · 모델 설정(meta.json, v5_final.json) · 실행 로그(dev 선택 · 최종 시험 · 확인 필요 평가 · LOW_CONF 스윕) · 최종 시험 결과(v5_final_test_result.json, 예측 파일) · legacy_v1 데이터와 학습 기록. 클래스별 정밀도 · 재현율 · F1과 학습 데이터 분포 통계는 이 파일들로 다시 계산한 값.